NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR EMBEDDED SYSTEMS

notes.arpanadhikari7.com.np

LAB REPORT 8: VHDL CODE FOR ENCODER & DECODER

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

Available on notes.arpanadhikari7.com.np

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20th Nov, 2025	
2.	VHDL Code (OR & NOT Gate) using testbench	23rd Nov	
3.	Universal Gates and Special Gates	30th Nov	
4.	VHDL Code for Adder	7th Dec	
5.	VHDL Code for Subtractor	14th Dec	
6.	VHDL Code for Multiplexer	21st Dec	
7.	VHDL Code for De-Multiplexer	4th Jan, 2026	
8.	VHDL Code for Decoder and Encoder	18th Jan	

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

LAB 8: VHDL CODE FOR DECODER AND ENCODER

Objective

The objective of this experiment is to **design and implement a 3:8 Decoder and an 8:3 Encoder using VHDL**. The aim is to understand the working of **combinational logic circuits** that perform **data expansion and data compression**. This experiment also helps in learning how to describe hardware using **VHDL**, simulate the designs, and verify their functionality using **truth tables**.

Theory

3:8 Decoder

A **3:8 decoder** is a **combinational logic circuit** that converts **3 binary input lines into 8 output lines**. For each unique input combination, **only one output line is activated (logic HIGH)**, while all other outputs remain LOW. Thus, it performs **binary-to-one-hot code conversion**.

A 3:8 decoder has **three inputs** (A_2, A_1, A_0) and **eight outputs** (Y_0 to Y_7). The active output corresponds to the **binary value of the input**. Since the decoder is a combinational circuit, its output depends only on the **present input values**.

3:8 decoders are widely used in **memory address decoding, instruction decoding, data selection, and digital control circuits**.

Truth Table of 3:8 Decoder

A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

notes.arpanadhikari7.com.np

8:3 Encoder

An **8:3 encoder** is a **combinational logic circuit** that converts **eight input lines into a 3-bit binary output**. Only **one input is assumed to be active at a time**, and the output represents the **binary equivalent of the active input line**. Hence, an encoder performs **data compression**.

An 8:3 encoder has **eight inputs** (D_0 to D_7) and **three outputs** (A_2, A_1, A_0). When a particular input line is HIGH, the encoder produces the corresponding 3-bit binary code at the output. The circuit has **no memory** and does not require a clock signal.

8:3 encoders are commonly used in **keyboard encoding, priority encoding, interrupt systems, and digital communication circuits**.

Available on notes.arpanadhikari7.com.np

Truth Table of 8:3 Encoder

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Encoder and Decoder Using VHDL

Both the **3:8 decoder and 8:3 encoder** are implemented in VHDL as **combinational circuits** using **case or conditional statements**. VHDL allows easy modeling, simulation, and verification of these circuits before hardware implementation, making it suitable for digital system design.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3:8 DECODER

DATAFLOW:

```
library IEEE;
use IEEE.std_logic_1164.all;

Entity arp_decoder is
port(
  I : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arp_decoder;

architecture arch_arpdecoder of arp_decoder is
begin
  Y(0) <= (not I(2)) and (not I(1)) and (not I(0));
  Y(1) <= (not I(2)) and (not I(1)) and (    I(0));
  Y(2) <= (not I(2)) and (    I(1)) and (not I(0));
  Y(3) <= (not I(2)) and (    I(1)) and (    I(0));
  Y(4) <= (    I(2)) and (not I(1)) and (not I(0));
  Y(5) <= (    I(2)) and (not I(1)) and (    I(0));
  Y(6) <= (    I(2)) and (     I(1)) and (not I(0));
  Y(7) <= (    I(2)) and (     I(1)) and (    I(0));
end arch_arpdecoder;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

BEHAVIORAL:

```
library IEEE;
use IEEE.std_logic_1164.all;

Entity arp_decoder is
port(
  I : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arp_decoder;

architecture arch_arpdecoder of arp_decoder is
begin
  process(I)
    begin
      case I is
      when "000" => Y <= "00000001";
      when "001" => Y <= "00000010";
      when "010" => Y <= "00000100";
      when "011" => Y <= "00001000";
      when "100" => Y <= "00010000";
      when "101" => Y <= "00100000";
      when "110" => Y <= "01000000";
      when "111" => Y <= "10000000";
      when others => Y <= "00000000";
      end case;
    end process;
end arch_arpdecoder;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
library IEEE;
use IEEE.std_logic_1164.all;

Entity arp_decoder is
port(
  I : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end arp_decoder;

architecture arch_arpdecoder of arp_decoder is
begin
  process(I)
    begin
      case I is
      when "000" => Y <= "00000001";
      when "001" => Y <= "00000010";
      when "010" => Y <= "00000100";
      when "011" => Y <= "00001000";
      when "100" => Y <= "00010000";
      when "101" => Y <= "00100000";
      when "110" => Y <= "01000000";
      when "111" => Y <= "10000000";
      when others => Y <= "00000000";
      end case;
    end process;
end arch_arpdecoder;
```

notes.arpanadhikari7.com.np

TESTBENCH:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpdecoder is
end test_arpdecoder;

architecture arp_behavioral of test_arpdecoder is
component arp_decoder
port(
 I : in std_logic_vector(2 downto 0);
 Y : out std_logic_vector(7 downto 0)
```


Available on notes.arpanadhikari7.com.np

```
);
end component;

signal I : std_logic_vector(2 downto 0);
signal Y : std_logic_vector(7 downto 0);

begin

UUT: arp_decoder port map (
   I => I,
   Y => Y
);

stim_proc : process
begin
  I <= "000"; wait for 100 ps;
  I <= "001"; wait for 100 ps;
  I <= "010"; wait for 100 ps;
  I <= "011"; wait for 100 ps;
  I <= "100"; wait for 100 ps;
  I <= "101"; wait for 100 ps;
  I <= "110"; wait for 100 ps;
  I <= "111"; wait for 100 ps;
end process;
end arp_behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpdecoder is
end test_arpdecoder;

architecture arp_behavioral of test_arpdecoder is
component arp_decoder
port(
  I : in std_logic_vector(2 downto 0);
  Y : out std_logic_vector(7 downto 0)
);
end component;

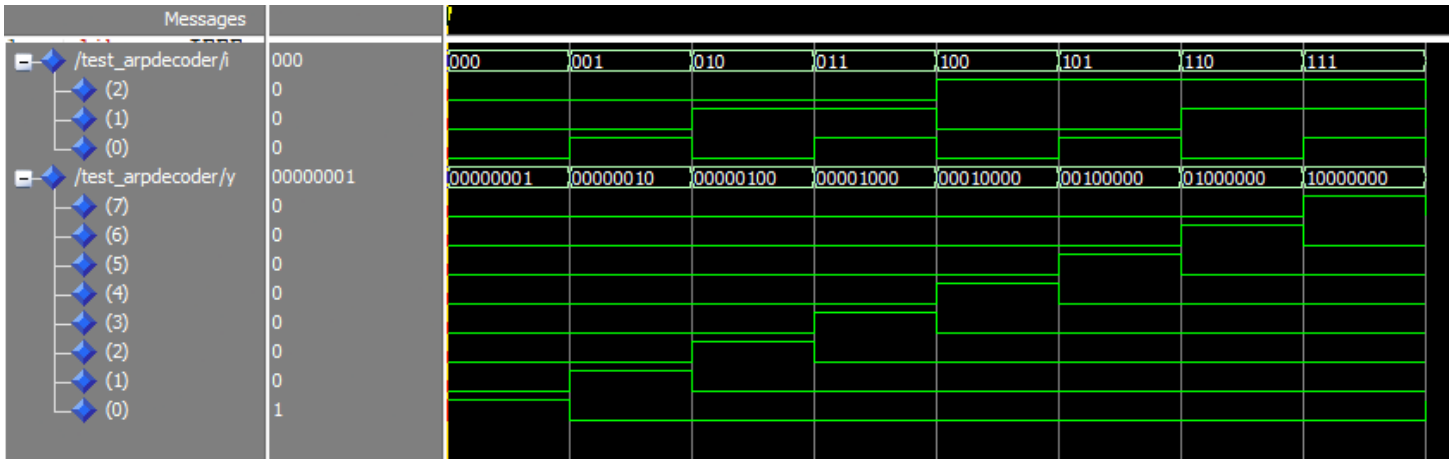
signal I : std_logic_vector(2 downto 0);
signal Y : std_logic_vector(7 downto 0);

begin
UUT: arp_decoder port map (
    I => I,
    Y => Y
);
stim_proc : process
begin
  I <= "000"; wait for 100 ps;
  I <= "001"; wait for 100 ps;
  I <= "010"; wait for 100 ps;
  I <= "011"; wait for 100 ps;
  I <= "100"; wait for 100 ps;
  I <= "101"; wait for 100 ps;
  I <= "110"; wait for 100 ps;
  I <= "111"; wait for 100 ps;
end process;
end arp_behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

WAVEFORM:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

8:3 ENCODER

DATAFLOW:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_encoder is
port(
  D : in  std_logic_vector(7 downto 0);
  A : out std_logic_vector(2 downto 0)
);
end arp_encoder;

architecture arch_arpencoder of arp_encoder is
begin
  A(2) <= D(4) or D(5) or D(6) or D(7);
  A(1) <= D(2) or D(3) or D(6) or D(7);
  A(0) <= D(1) or D(3) or D(5) or D(7);
end arch_arpencoder;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

BEHAVIORAL:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arp_encoder is
port(
  D : in  std_logic_vector(7 downto 0);
  A : out std_logic_vector(2 downto 0)
);
end arp_encoder;

architecture arch_arpencoder of arp_encoder is
begin
  process(D)
  begin
    case D is
      when "00000001" => A <= "000";
      when "00000010" => A <= "001";
      when "00000100" => A <= "010";
      when "00001000" => A <= "011";
      when "00010000" => A <= "100";
      when "00100000" => A <= "101";
      when "01000000" => A <= "110";
      when "10000000" => A <= "111";
      when others     => A <= "000";
    end case;
  end process;
end arch_arpencoder;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

TESTBENCH:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_arpencoder is
end test_arpencoder;

architecture arp_behavioral of test_arpencoder is

component arp_encoder
port(
  D : in  std_logic_vector(7 downto 0);
  A : out std_logic_vector(2 downto 0)
);
end component;

signal D : std_logic_vector(7 downto 0);
signal A : std_logic_vector(2 downto 0);

begin

UUT: arp_encoder port map (
  D => D,
  A => A
);

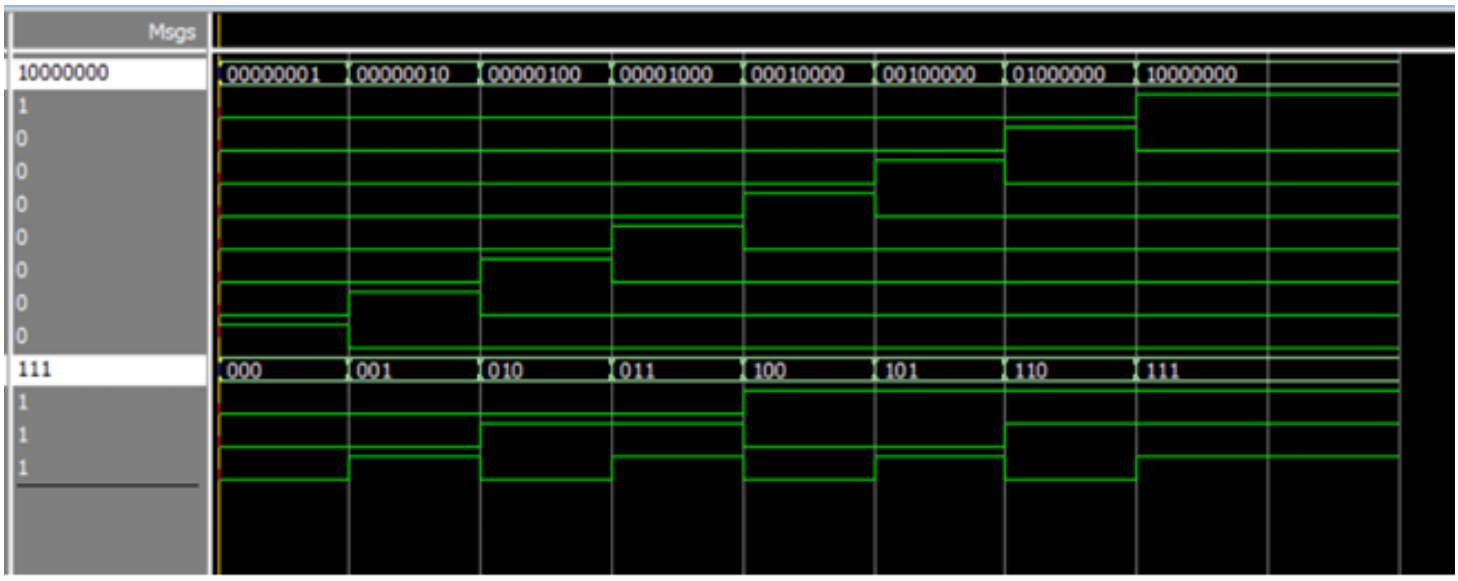
stim_proc : process
begin
  D <= "00000001"; wait for 100 ps;
  D <= "00000010"; wait for 100 ps;
  D <= "00000100"; wait for 100 ps;
  D <= "00001000"; wait for 100 ps;
  D <= "00010000"; wait for 100 ps;
  D <= "00100000"; wait for 100 ps;
  D <= "01000000"; wait for 100 ps;
  D <= "10000000"; wait for 100 ps;
end process;

end arp_behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

WAVEFORM:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Result

The **3:8 Decoder** and **8:3 Encoder** were successfully designed and simulated using **VHDL**. The simulation outputs matched the expected **truth tables**. The decoder activated the correct single output for each input combination, and the encoder generated the correct binary code for each active input.

Conclusion

The experiment successfully demonstrated the design and verification of **combinational circuits** using **VHDL**. The **3:8 decoder** performed correct data expansion, and the **8:3 encoder** performed correct data compression. This experiment helped in understanding **dataflow and behavioral modeling** in VHDL.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np